Capstone Project:

Darkstew Hollow

Joseph Coats, Owen Manley, Kyle Walbring

CSC496 - Capstone II

Quincy University

Table of Contents

Abstract

We are producing a video game called Darkstew Hollow, a dungeon crawler styled adventure that aims to have a fun and meaningful experience. Even for recurring players, the game will stay interested in the game's unpredictable mines and caverns as they find various treasures and enemies. Among these findings will be various stones present on Earth, with enlightening descriptions on their properties and origin. This adventure will be inspired by other games like it, though they are not solely invested in mining. Although there are some limitations and more notable challenges, many of these challenges are intrinsically tied to our hopes of pushing the boundaries of our game’s mechanics and functionality. Our choice of game engine is specifically designed to aid beginner developers, and our graphics will be produced in a free to use software called Krita [5]. Overall, this will allow us to gain experience working as a team of developers and making the most of our own unique strengths, with the result being a well-designed interactive experience for anybody to try.

Project Description

Our primary goal for our capstone project is to create an original and interactive multimedia experience that holds exploration, battle, and educational features for our target players. It will feature rogue-like elements that are similar to many RPGs (role-playing games). New mechanics will be introduced at a steady rate for the players to familiarize themselves with throughout their playthrough. Specifically, the audience is aimed to be people who are new to video games while still giving seasoned players a challenge. In the meantime, a unique audience will be targeted for our geological systems in-game. People who are interested in a simple story and want to have fun while fighting a variety of imaginative monsters and hunting for underground rewards will enjoy the definitive experience that we intend to provide. Overall, our highest concern lies with fun above all since an educational message can often be diluted or dismissed by the uninterested person.

Furthermore, the limitations that we have pertaining to our game are our ideas such as pets and cosmetics versus our time constraints. We have many features and challenges we want to try, but must assess and prioritize the few with the most value. Meanwhile, any other limitations have yet to be encountered or acknowledged due to our experience in computer science.

Additionally, our project will be inspired by a game called Stardew Valley [2] which is mainly themed around farming and community. Although this game does have its own mines and monsters, it is merely a portion of the whole experience that is not heavily expanded upon. Our

project will extend this concept by adding traps, stones, vast mine shafts/dungeons, and more features that are planned to be implemented if enough time is given in the future.

Moreover, the player engages the game as a humble squire. Lightly armored with his aged shortsword, the townsfolk will arm him with a pickaxe to delve into the corrupted mines below. With each level going deeper, the environment will increase in difficulty ranging from complicated layout of the levels, difficulty in the enemies faced, to hazards pertaining to traps. However, as the caves below increase in difficulty, the rewards increase in abundance as well. If the player is defeated within the mines, they will awaken back in the nearby village of Ironhaven. In addition, the goal of the player is to gather resources underground to restore the damaged town. It is made clear by the townsfolk that the monsters of the mines have once emerged from the mines to destroy the town. But, the motives of the creatures below are not clear and we plan to answer that question in the game's finale. Meanwhile, the player will oversee the reconstruction of ruined buildings which include the blacksmith's forge, the armorer, and the alchemist's apothecary (more in development), each with its own unique upgrades to the player. So far, the blacksmith will allow the player to upgrade their pickaxe and sword to make it easier to break resources or slice through the monsters. The armorer will provide the player with stronger armor, making them more resistant to being damaged by the creatures within the mines. And finally, the alchemist will provide potions that can heal the player or potentially grant them temporary abilities.

Ultimately, we are fully expecting to come across some challenges while we develop our capstone project. Whilst not saying that the team altogether isn't confident in the outcome of the project, but rather explaining with due confidence that we are prepared to make the necessary adjustments in the event that such a challenge arises. Challenges could come across in

development that would greatly increase the difficulty of completing the project. Therefore, some challenges that we are keeping at the front of our minds include factors like procedural generation and getting the Godot [3] engine to be compatible with C#.

Currently, Godot has not done so, thus we will be using the native scripting language. Fortunately, Godot is not only beginner-friendly but well documented which makes it an effective choice for our first project. Graphical elements such as the terrain and character sprites will be made in Krita; we decided to use this software because it is open source, granting access to various community-made tools that can be added to it. Additionally, they are both known well enough that there is an abundance of tutorials available online for the team to learn from.

By creating this game, we will be able to code in the native Godot scripting language, which has similarities to other coding languages like Python and Javascript. Furthermore, this will strengthen our knowledge of these languages since we will be able to relate the new language back to something we already know from having made this project. Additionally, this project will prepare us for working in teams on projects instead of individually working on an assignment like in the past. Being prepared to work in teams will make us better prepared for working in computer science fields, as we will learn how to work together, use our strengths, and manage large projects such as this.

Literature Review

Stardew Valley [2] is an indie game made by Eric Barone, more professionally known as ConcernedApe. It is a farming simulation game in which the player inherits a farm that is now overrun with weeds and rocks from their now-deceased grandfather. The main objective of the game is to complete the Community Center, which like your newly inherited farm, has seen better days. Through tending to your crops, fishing in the local waters, collecting resources from the nearby mines, slaying enemies, and befriending the NPCs residing in Pelican Town, the player advances towards achieving 100% completion of the game. Hollow Knight [9] is a game where the player controls a nameless hero, known only as the Knight. Exploring the fallen kingdom of Hallownest, the player must kill hostile enemies and various bosses scattered throughout the underground kingdom. As they explore and defeat certain bosses, the player will unlock more upgrades and abilities, such as a stronger weapon or the ability to double jump. Realm of the Mad God [8] is a pixelated, top-down, MMORPG that allows the player to control any kind of archetype of their choosing in the game. Coming out in 2012, this game bolstered a substantial popularity for its unique spin on the MMO formula. Furthermore, the game consists of players going around different realms to level up and get stronger. After a period of time, all players on the server will be transferred to “The Realm of the Mad God” where they will need to overcome a difficult dungeon together to defeat the final boss. There are multiple classes to choose from, and many skills/weaponry to obtain to make oneself stronger for the foes presiding in the realms. Overall, what makes this game unique is its unique art style, mechanics, and

community that held its own while games such as World of Warcraft were dominating the state of MMOs.Godot [3] is an intuitive game engine designed for both new and experienced developers. It boasts several features including a node system, community-driven elements, and the use of scripting and high-level API for these nodes.

Functional Requirements

FUNCTIONAL REQUIREMENTS IMPLEMENTED

- The game should be able to offer single-player.
- The player should be able to interact with the character using a keyboard/mouse or a controller.
- The game should allow certain UI elements such as a health bar, a hotbar, a main menu, an inventory, options for sound and graphics, as well as dialogue boxes.

FUNCTIONAL REQUIREMENTS NOT IMPLEMENTED CURRENTLY

- The player should be able to mine nodes and find artifacts in levels to receive the ability to rebuild back in Ironhaven.
- The player should have a variety of different solutions to counter enemies (melee, magic, ranged).
- The player should have the ability to upgrade equipment back in Ironhaven once requirements are met.
- The game should allow a log so players know how many materials are needed to construct a building when delving into the mines (toggleable).
- The game should have a tutorial in the beginning to teach players the game.

NON-FUNCTIONAL REQUIREMENTS IMPLEMENTED

- UI should be simple and intuitive to navigate.

NON-FUNCTIONAL REQUIREMENTS NOT IMPLEMENTED CURRENTLY

- The game should be able to run on low-end PCs like laptops with 8GB of RAM.

- The game should be compatible with Windows.
- The game should be able to run on a higher frame rate average (60).

More functional and non-functional requirements are to be implemented in the future if they come up. For additional functional requirements, these can include different kinds of tools able to be used to counter foes. Comparatively, this differs from solutions to counter enemies because solutions refer to strategies while tools refer to different items corresponding to the solution. For non-functional requirements, this can refer to being able to perform on different platforms like consoles which is only an example; however, it could be a potential implementation if time ordains it. As of present, these are the requirements currently documented officially.

Use Case Modeling

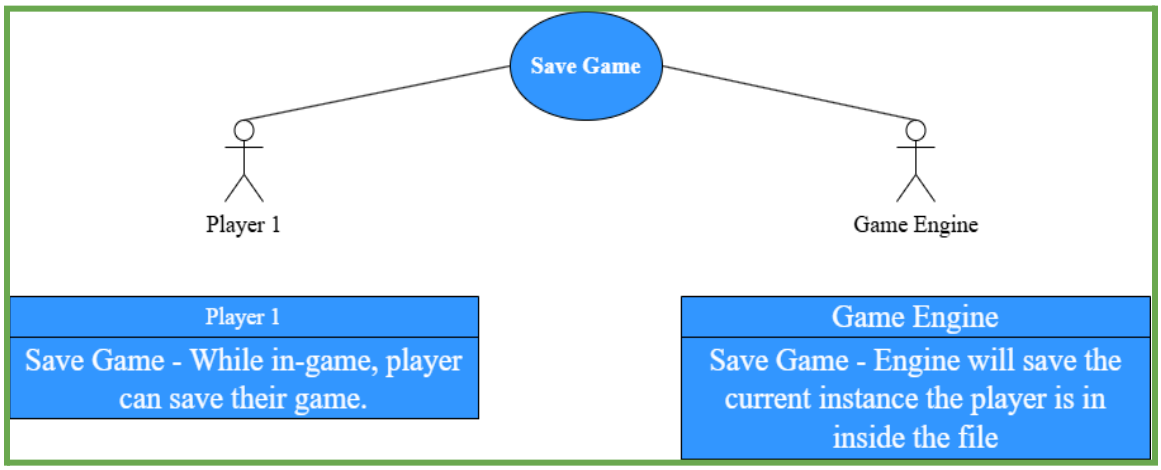

Figure 1

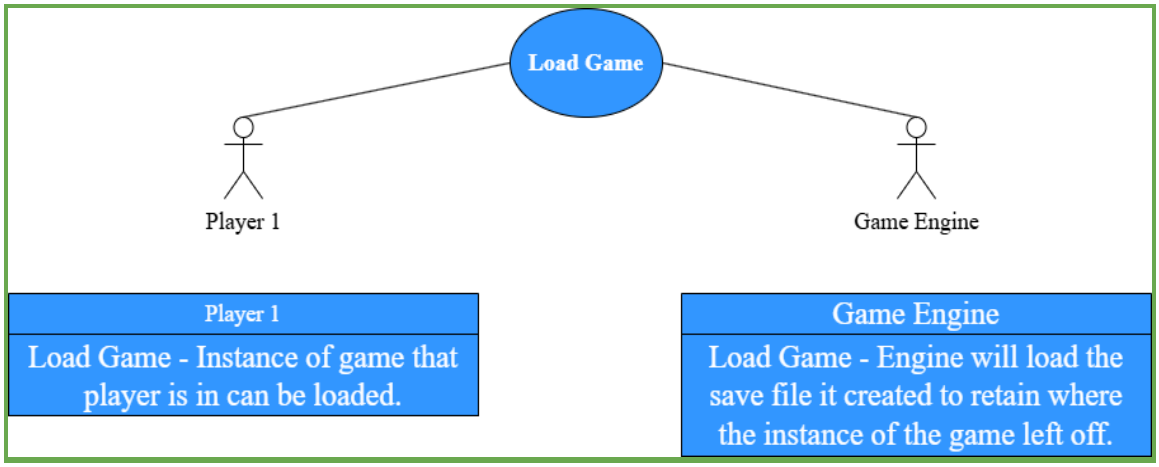

Figure 2

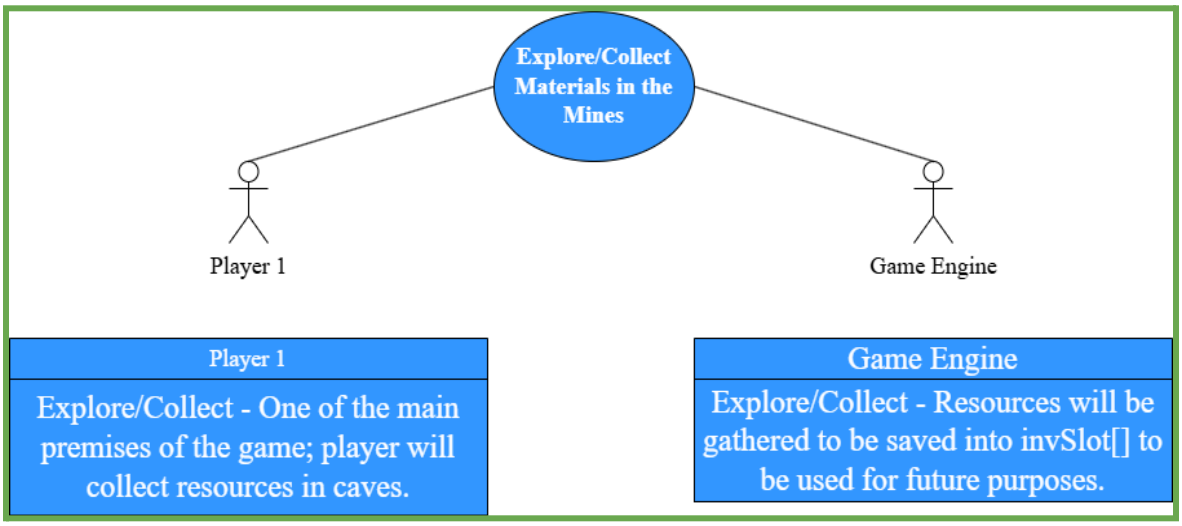

Figure 3

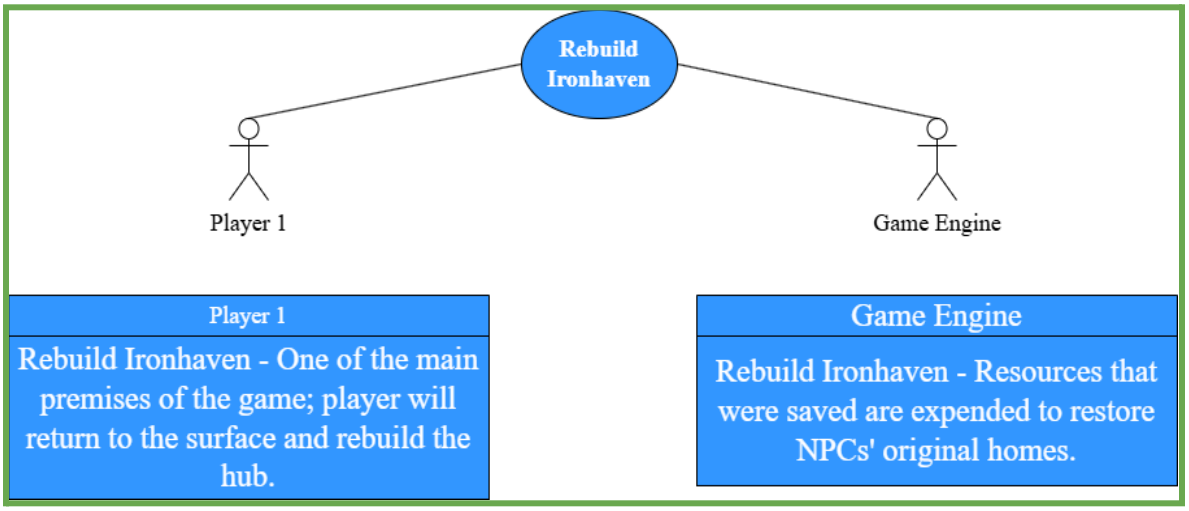

Figure 4

UML Diagram

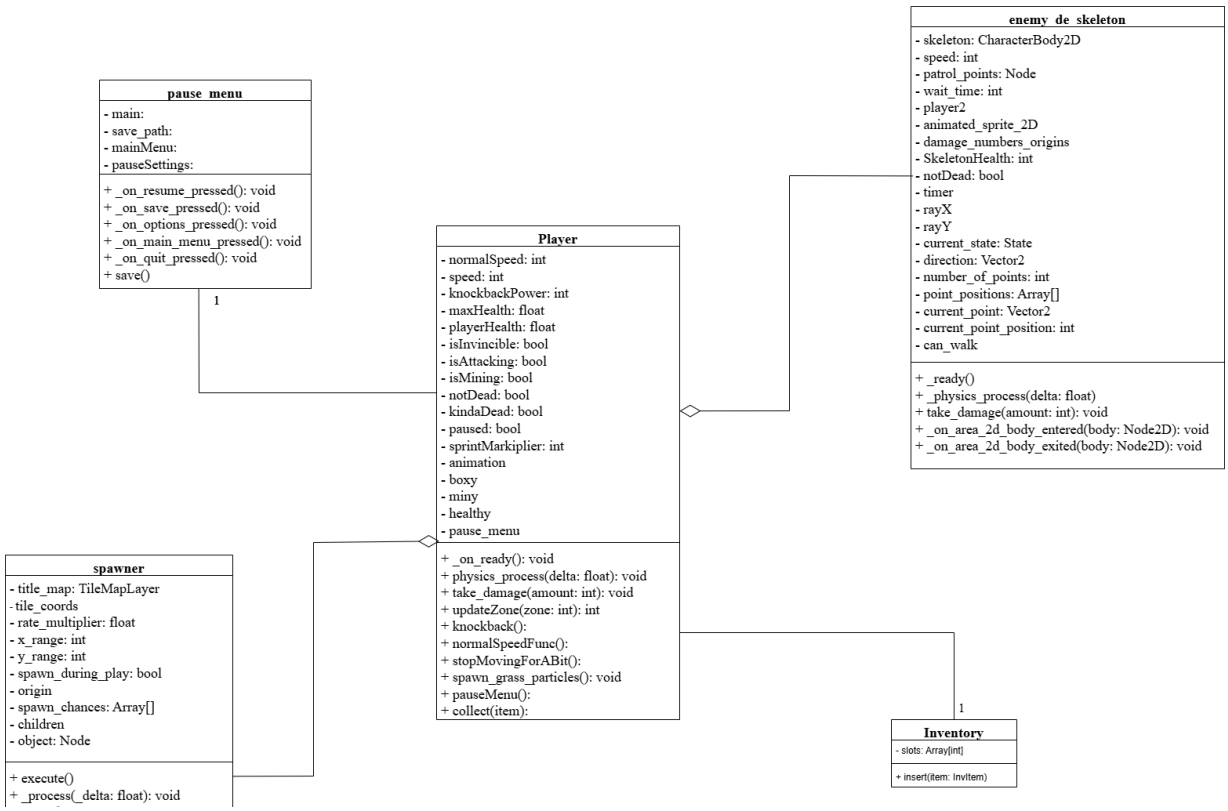

Figure 5

Player.gd - Class that allows the user responsibility for any action in the game. Key methods on_ready() because it is responsible for displaying health, take_damage() which drains health, pauseMenu() for pausing the game, and collect() being responsible for putting items in the inventory.

Enemy_de_skeleton - The class that handles the “skeleton” enemy in Darkstew Hollow. It is aggregation because Player.gd would still function without this class. Key methods are _ready() which handles its patrolling and on_area_2d_body_entered because it is responsible for dealing and taking damage.

Inventory.gd - Is responsible for building the foundation for the inventory on the player. Inventory associates with Player.gd using one call to the class. Insert() is its key function and it deals with inserting items that are picked up into the player’s inventory.

Pause_menu - Pauses the game while in session. Every method is important here because it handles the keeping of the game. For instance, the user can quit without saving, decide to save, or go into more detailed options.

Spawner.gd - Handles the spawning of enemies in Darkstew Hollow. In the class, _pick_random_pos is responsible for choosing a random location on the grid for a spawn. Meanwhile spawn() handles the actual spawning of enemies in the random location chosen. Both are the most important methods here.

System Architecture

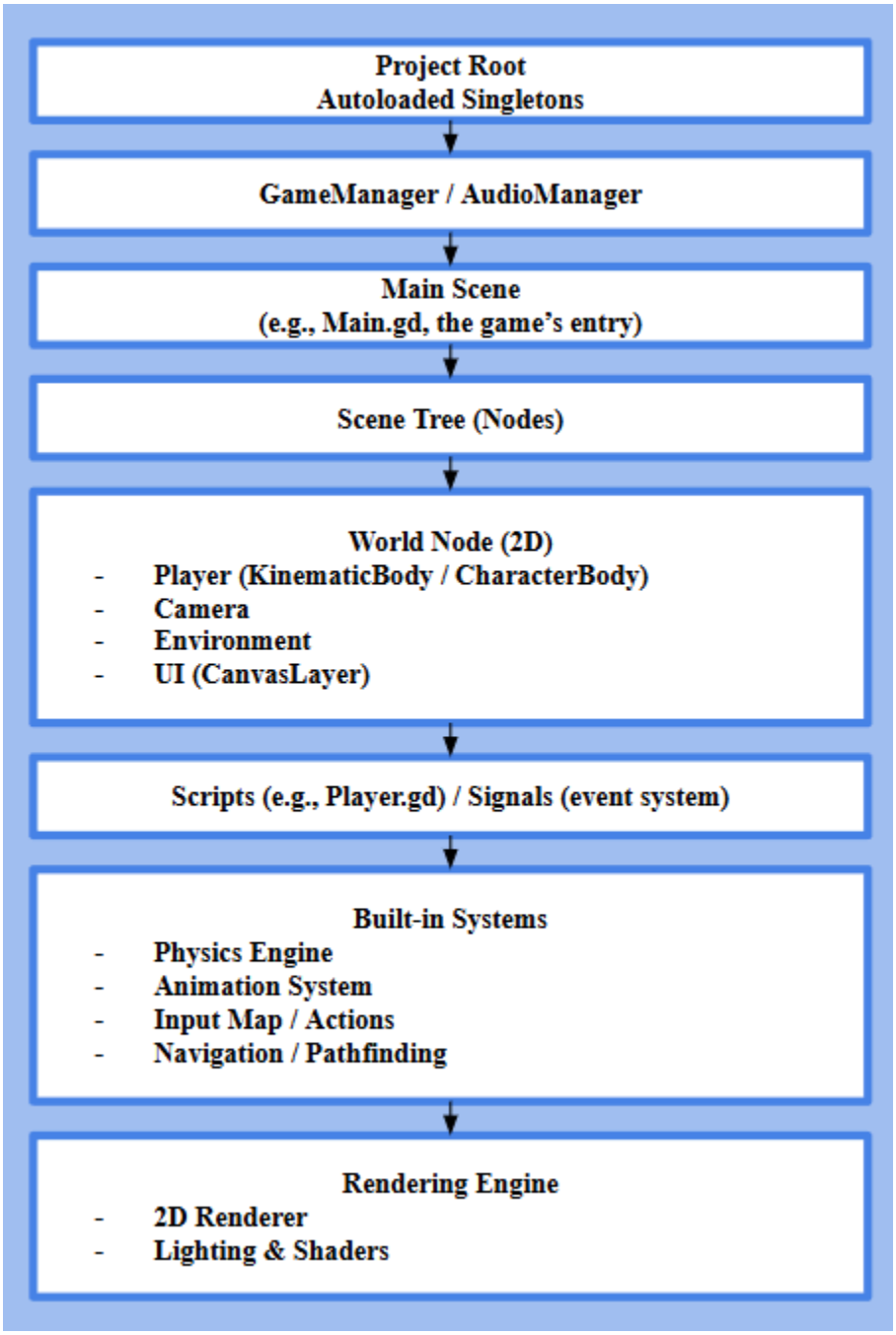

Figure 6: The system architecture that is enforced by Godot.

Data Design

The data structures that we use in our game Darkstew Hollow are files. These files are written in binary, serving to save the state of certain attributes of our game, which includes the scene that the player is currently in, the previous scene that the player was in, and other player data such as the player's health, their current x position, their current y position, and their inventory. In total, these save files take up roughly 100 bytes.

Screenshots

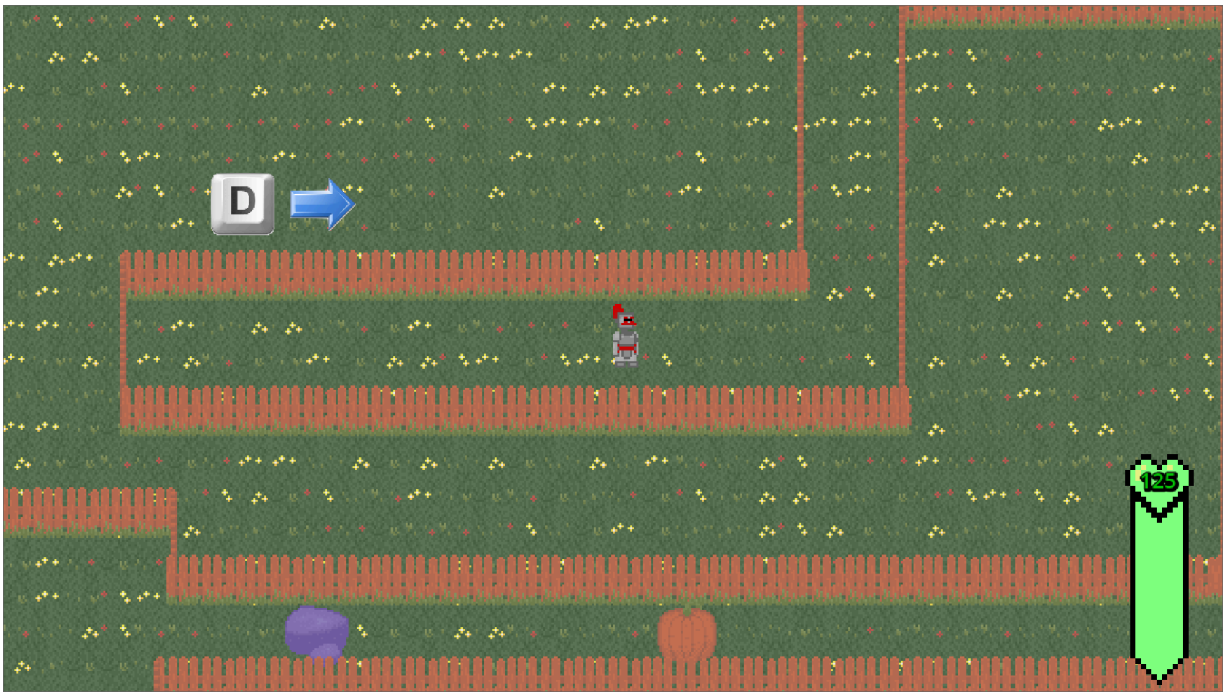

Figure 7

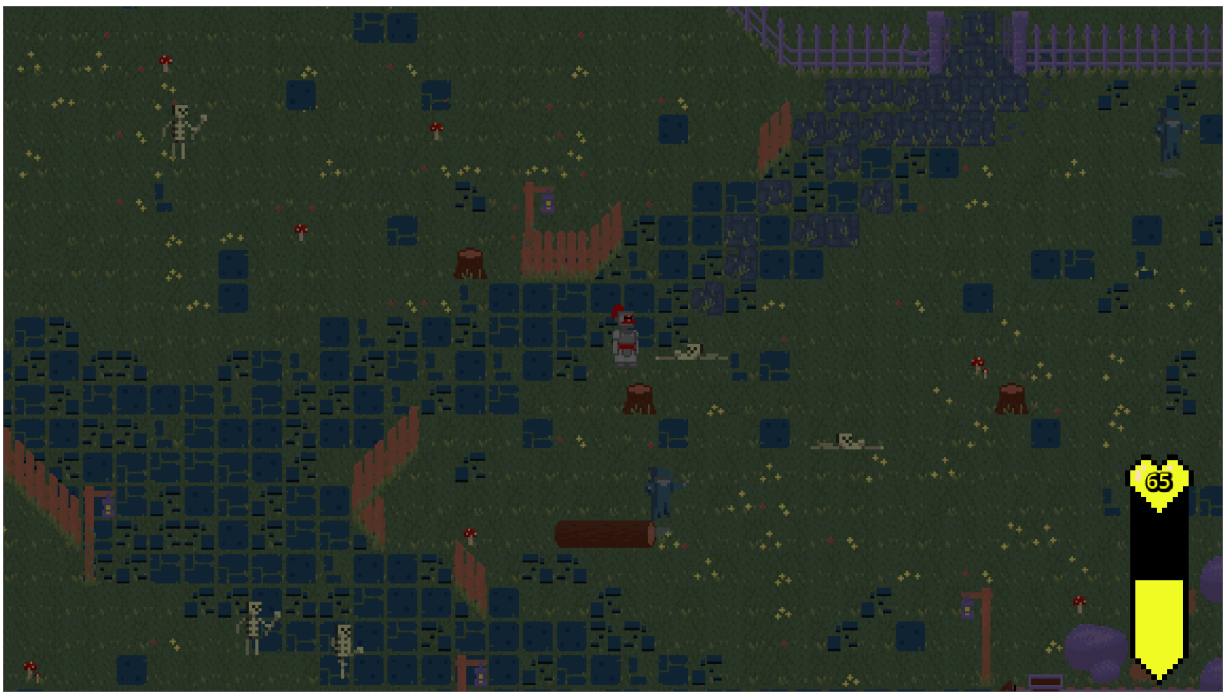

Figure 8

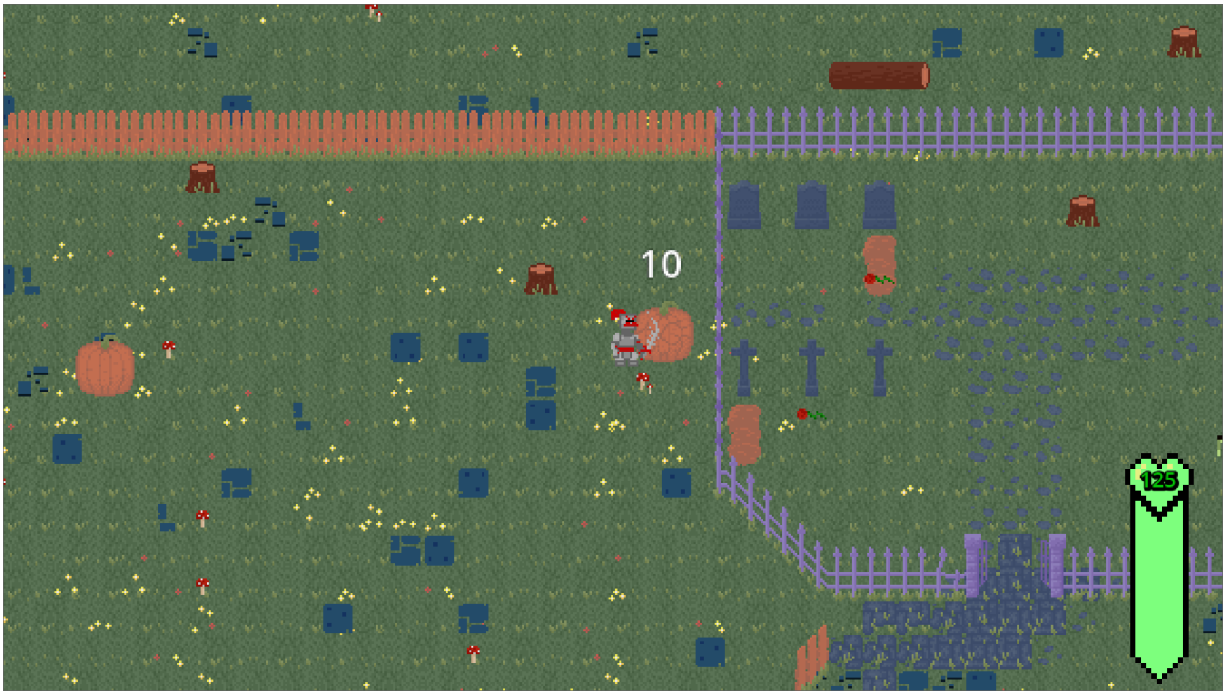

Figure 9

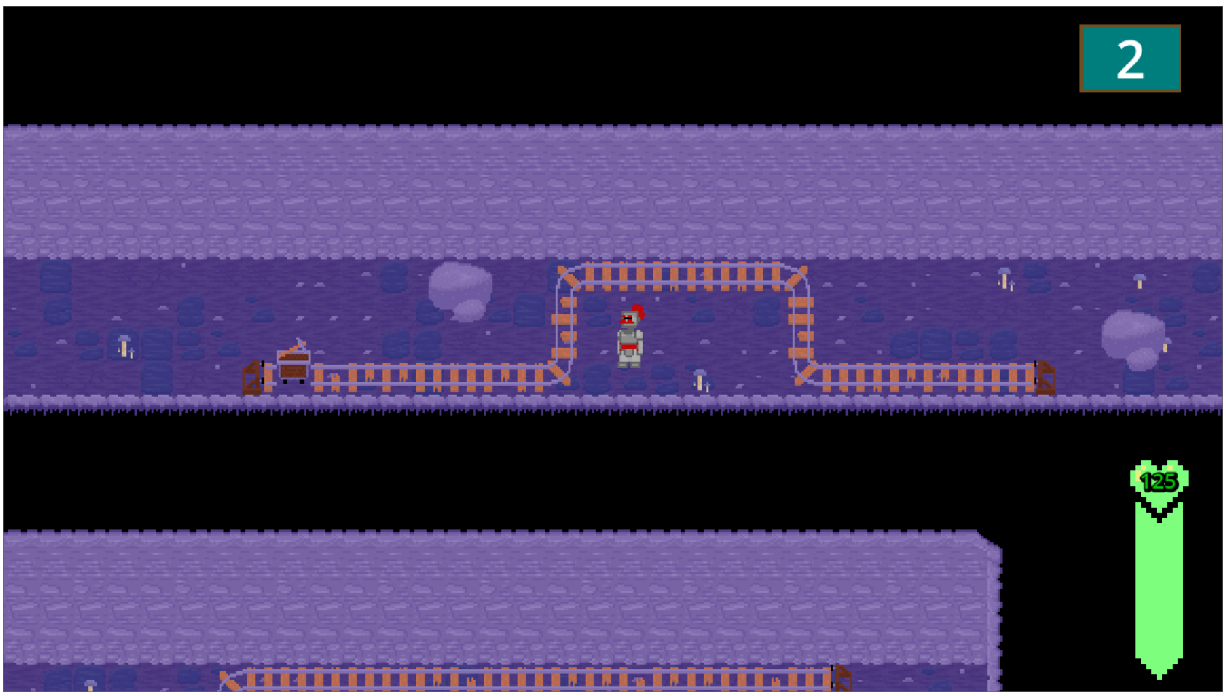

Figure 10

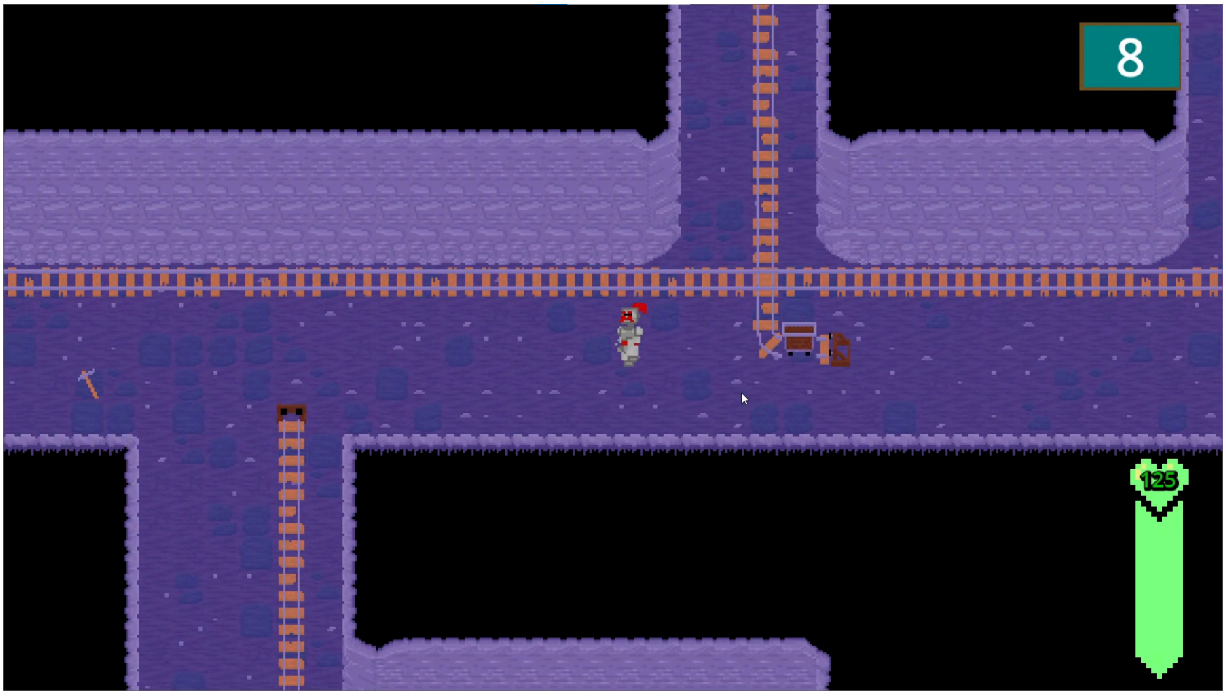

Figure 11

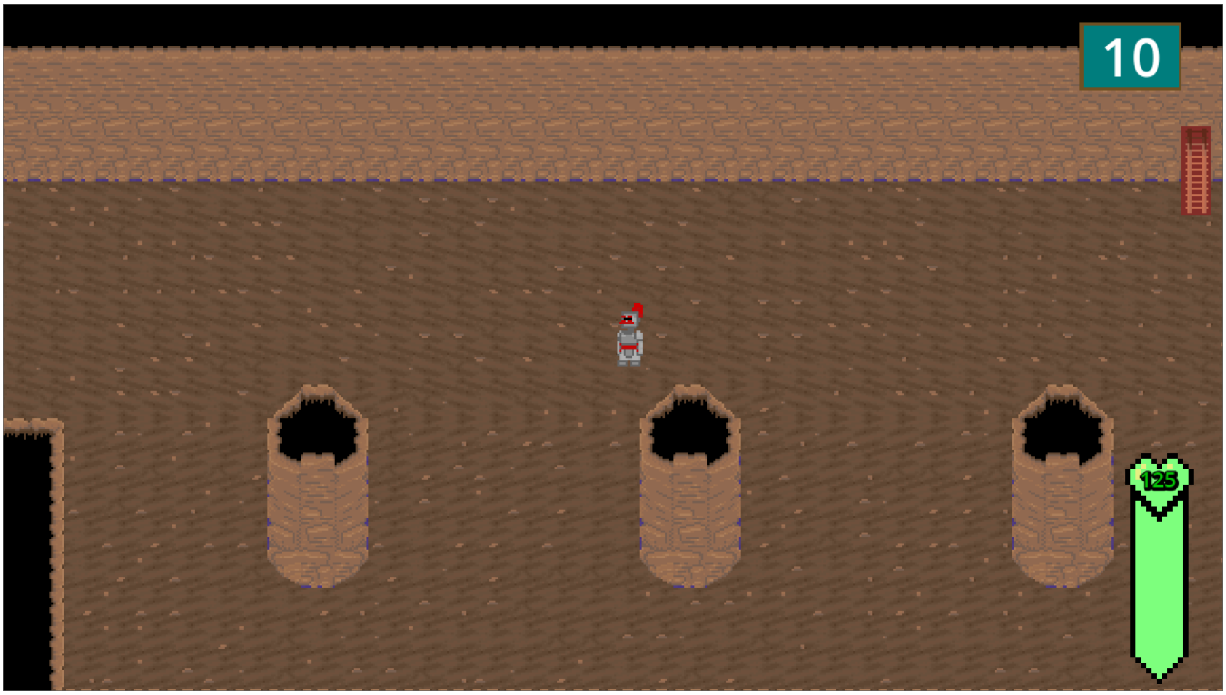

Figure 12

Glossary of Terms

Buff - A temporary increase to a character's stats. This can apply to the player or enemies and can affect stats such as movement speed or health.

Cutscene - A scene that is not interactive that serves to progress the game's story

Critical hit - An attack that does considerably more damage than a normal attack. Critical hits often have a percent chance of occurring instead of a standard attack.

Dungeon crawler - A genre of games in which players navigate a dungeon, battling enemies and collecting loot as they progress.

HP - "Health points" or "hit points", refers to the total amount of damage that a character can take before they are killed

Minigame - A short game that is contained within a larger game. An example includes the lockpicking minigame in Skyrim [1].

Min-maxing - A way of playing a game, commonly RPGs, where the player strives for perfect optimization of their gameplay. An example in Stardew Valley [2] is to pause the game as soon

as possible after descending a floor in the mines, as this pauses time in-game while the floor loads.

NPC - “Non-player character”, refers to a character in the game that is not controlled by the player. Typically takes on the role of adding lore to the game or adding transactional experiences such as serving as a shop owner or giving the player quests.

Procedural Generation - A technique in which elements of a game are generated in real-time from a fixed set of variables or parameters.

RPG - “Role-playing game”, refers to a story-driven game that is centered around a main character, usually the player character.

Future Work

We would like to continue our work on Darkstew Hollow after our graduation. There are still many features that we would like to implement and add. Some examples of these features include expanding on the story and adding more areas, NPCs, and interactable options to assist with this goal. Additionally, we would like to improve certain features that while have already been implemented, have not been optimized. This includes improving the enemy and NPC pathing to better navigate objects in the world, and also add procedural generation to the caves instead of just having ten premade levels for every theme.

Conclusion

In conclusion, Darkstew Hollow stands as a hallmark for creative and collaborative achievement in our first steps towards game development. Furthermore, our contributions provide a take on rogue-like and roleplaying mechanics unique to us. Providing geological and archeological elements in the project also give unique elements that provide a unique task to players as they unravel the growing narrative behind Darkstew Hollow as they unearth artifacts deep within the Ironhaven Mines. The use of the Godot game engine and Krita provided us with open-source tools that supported the growth of our knowledge as developers. Conversely, we went with this decision so that we wouldn't have to worry about the unnecessary monetization and complex management of Unity while also being able to create our very own assets with little trouble using Krita. Our node based structure, AI pathing, procedural generation, and mechanics for the player showcase the technical and conceptual contributions that we provided. However, the project also faced limitations when in development. Instances such as mining upgrades being incomplete, different styles for combat, learning a game engine, and learning GDScript are what provided the largest challenges when in development. Despite the obstacles we faced, Darkstew Hollow demonstrates our capabilities when it comes to large-scale development projects, adapting with each other as a team, and providing an engaging prototype for our game.

Works Cited

1. Bethesda. “The Elder Scrolls | Home.” *Elderscrolls.bethesda.net*, elderscrolls.bethesda.net/en. Accessed 23 Feb. 2025.
2. ConcernedApe. “Stardew Valley.” *Stardewvalley.net*, 2016, www.stardewvalley.net/.
3. Godot Engine. “Godot Engine - Free and Open Source 2D and 3D Game Engine.” *Godotengine.org*, 2019, godotengine.org/.
4. Holfeld, Julian. “On the Relevance of the Godot Engine in the Indie Game Development Industry.” *ArXiv (Cornell University)*, vol. 2, no. 2401.01909, Cornell University, Dec. 2023, https://doi.org/10.48550/arxiv.2401.01909.
5. Krita Foundation. “Krita | Digital Painting. Creative Freedom.” *Krita.org*, 2019, krita.org/en/.
6. Manzur, Ariel, and George Marques. *Godot Engine Game Development in 24 Hours, Sams Teach Yourself*. Sams Publishing, 2018.
7. “PlayStation’s Ultimate List of Gaming Terms | This Month on PlayStation.” *PlayStation*, www.playstation.com/en-us/editorial/playstation-ultimate-gaming-glossary/. Accessed 30 Nov. 2024.
8. “Realm of the Mad God.” *Realmofthemadgod.com*, 2019, www.realmofthemadgod.com/. Accessed 24 Sept. 2019.
9. Team Cherry. “Hollow Knight.” *Hollow Knight*, Team Cherry, www.hollowknight.com/. Accessed 23 Feb. 2025.

10. Ternikov, Andrei. “Soft and Hard Skills Identification: Insights from IT Job Advertisements in the CIS Region.” *PeerJ Computer Science*, vol. 8, Apr. 2022, p. e946, https://doi.org/10.7717/peerj-cs.946.
11. Wikipedia. “Stardew Valley.” *Wikipedia*, 10 May 2020, en.wikipedia.org/wiki/Stardew_Valley.

Appendix A

breakable_big_rock.gd

```
extends Node2D

@onready var damage_numbers_origin = $DamageNumbersOrigin
@onready var health = 30 #rock health
@onready var notDead = true #box is alive

func _on_ready() -> void:
	$StaticBody2D/collide.set_deferred("disabled", false)

func take_damage(amount: int) -> void: #amount is the damage taken
	#print("Damage: ", amount) #prints the damage taken to the console
	health -= amount
	if(health > 0): #damage not kills the box
		DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical

	else:
		if(notDead == true):
			DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
			notDead = false
		$PropsCave.visible = false
		$DamageNumbersOrigin.visible = false
		$StaticBody2D/collide.set_deferred("disabled", true)
```

cave_sounds.gd

```
extends AudioStreamPlayer

const level_music = preload("res://Assets/Audio/2-07. Chargestone Cave.mp3")

func _play_music(music: AudioStream, volume = 0.0):
	if stream == music:
		return

	stream = music
	volume_db = volume
```

```
	play()

func play_music_level():
	_play_music(level_music)
```

DamageNumbers.gd

```
extends Node


func display_number(value: int, position: Vector2): #Vector2, is_critical: bool = false):
	var number = Label.new()
	number.global_position = position
	number.text = str(value)
	number.z_index = 5
	number.label_settings = LabelSettings.new()

	var color = "#FFF"
	#if is_critical:
		#color = "#B22"
	if value == 0:
		color = "#FFF8"

	number.label_settings.font_color = color
	number.label_settings.font_size = 18
	number.label_settings.outline_color = "#000"
	number.label_settings.outline_size = 1


	call_deferred("add_child", number)

	await number.resized
	number.pivot_offset = Vector2(number.size / 2)

	var tween = get_tree().create_tween()
	tween.set_parallel(true)
	tween.tween_property(
		number, "position:y", number.position.y - 24, 0.25
	).set_ease(Tween.EASE_OUT)
	tween.tween_property(
		number, "position:y", number.position.y, 0.5
	).set_ease(Tween.EASE_IN).set_delay(0.25)
	tween.tween_property(
		number, "scale", Vector2.ZERO, 0.25
	).set_ease(Tween.EASE_IN).set_delay(0.5)
```

```
	await tween.finished
	number.queue_free()
```

enemyHitBox.gd

```
class_name enemyHitBox
extends Area2D

@export var damage := 10

func _init()-> void:
	collision_layer = 4
	collision_mask = 0
```

enemyHurtBox.gd

```
class_name enemyHurtBox
extends Area2D

func _init() -> void:
	collision_layer = 0
	collision_mask = 4

func _ready() -> void:
	connect("area_entered", self._on_area_entered)

func _on_area_entered(hitbox: enemyHitBox) -> void:
	if hitbox == null:
		return

	if (owner.has_method("take_damage")):
		owner.take_damage(hitbox.damage)
```

enemy_de_skeleton.gd

```
extends CharacterBody2D

@export var player: CharacterBody2D
@export var skeleton : CharacterBody2D
@export var speed : int = 150

@onready var animated_sprite_2d = $AnimatedSprite2D
@onready var damage_numbers_origin = $DamageNumbersOrigin
@onready var SkeletonHealth = 30 #30
@onready var notDead = true #skeleton is alive
```

```
#var direction : Vector2 = Vector2.RIGHT
#var current_point : Vector2
var boo : bool = false #player not there
var rayX
var rayY

func _physics_process(delta: float) -> void:
	velocity.x = 0
	velocity.y = 0

	if (boo == true && notDead == true): #player is within the "vision area" and the enemy is not dead

		#makes the raycast arrow follow the player precisely
		rayX = player.global_position.x - skeleton.global_position.x
		rayY = player.global_position.y - skeleton.global_position.y
		$RayCast2D.target_position = Vector2(rayX, rayY)

		$Area2D.look_at(player.global_position)



		if ($RayCast2D.is_colliding()):
			var collider = $RayCast2D.get_collider()
			#$Area2D/CollisionPolygon2D.polygon[3].x = 60
			#$Area2D/CollisionPolygon2D.polygon[4].x = 60
			if (collider == player):
				#move the enemy to the player
				var moveX = player.global_position.x
				var moveY = player.global_position.y
				global_position = global_position.move_toward(Vector2(moveX, moveY), speed * delta)
				if(global_position == Vector2(moveX, moveY)):
					print("ehre")




		#var skeleX : float = skeleton.global_position.x
		#var playerX : float = player.global_position.x
		#var skeleY :float = skeleton.global_position.y
		#var playerY :float = player.global_position.y
```

```
                #print("skeleX: " + str(skeleX) + " + " + "playerX: " + str(playerX))
                #print("skeleY: " + str(skeleY) + " + " + "playerY: " + str(playerY))
                #velocity.x = move_toward(skeleX, playerX, delta)
                #print(velocity.x)
                #velocity.y = move_toward(skeleY, playerY, delta)
                #print(velocity.y)

                #var skeleX : float = skeleton.global_position.x
                #var playerX : float = player.global_position.x
                #var skeleY :float = skeleton.global_position.y
                #var playerY :float = player.global_position.y
                #print(global_position.x)
                #if(((playerX - skeleX) > 0) && ((playerY - skeleY) > 0)): #positive number, so
positive delta
                        #velocity.x = move_toward(skeleX, playerX, delta)
                        #velocity.y = move_toward(skeleY, playerY, delta)
                #elif(((playerX - skeleX) < 0) && ((playerY - skeleY) > 0)): #negative x
                        #velocity.x = move_toward(skeleX, playerX, -delta)
                        #velocity.y = move_toward(skeleY, playerY, delta)
                #elif(((playerX - skeleX) > 0) && ((playerY - skeleY) < 0)): #negative y
                        #velocity.x = move_toward(skeleX, playerX, delta)
                        #velocity.y = move_toward(skeleY, playerY, -delta)
                #elif(((playerX - skeleX) < 0) && ((playerY - skeleY) < 0)): #negative x & y
                        #velocity.x = move_toward(skeleX, playerX, -delta)
                        #velocity.y = move_toward(skeleY, playerY, -delta)

        move_and_slide()

func take_damage(amount: int) -> void: #amount is the damage taken
        #print("Damage: ", amount) #prints the damage taken to the console
        speed = 0
        SkeletonHealth -= amount
        if(SkeletonHealth > 0): #damage kills the box
                DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
                #make so no takey damage here
                $myHurtbox/CollisionShape2D.set_deferred("disabled", true)
                await get_tree().create_timer(.5).timeout
                speed = 150
                await get_tree().create_timer(.5).timeout
                $myHurtbox/CollisionShape2D.set_deferred("disabled", false)
        else: #damage is kil
                if(notDead == true):
```

```
                        DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
                        notDead = false
                animated_sprite_2d.play("dies") #wont work, idk why???? #also this sprite is
ONLY the skeleton, not the whole spritesheet
                $DamageNumbersOrigin.visible = false
                $myHurtbox/CollisionShape2D.set_deferred("disabled", true)
                $enemyHitBox/CollisionShape2D.set_deferred("disabled", true)
                $CollisionShape2D.set_deferred("disabled", true)
                $Area2D/CollisionPolygon2D.set_deferred("disabled", true)
                $RayCast2D.enabled = false


#func look_for_player():
        #if($Node2D/RayCast2D.is_colliding()):
                #var collider = $Node2D/RayCast2D.get_collider()
                #if collider == player:
                        ##chase_player()
                        #print("chasing")

func _on_area_2d_body_entered(body: Node2D) -> void:
        if (body.is_in_group("Player") && notDead == true):
                print("body entered")
                boo = true

func _on_area_2d_body_exited(body: Node2D) -> void:
        if (body.is_in_group("Player") && notDead == true):
                print("body leaves (like from trees)")
                boo = false
                #$Area2D/CollisionPolygon2D.polygon[3].x = 90
                #$Area2D/CollisionPolygon2D.polygon[4].x = 90
```

health_bar.gd

```
extends CanvasLayer

@onready var healthBar = $HealthBar/Health
@onready var healthLabel = $HealthBar/HealthLabel

func update_health(playerHealth, maxHealth):
        healthLabel.text = str(playerHealth)
        if(((playerHealth / maxHealth) * 100) <= 10):
                healthBar.frame = 1
        elif(((playerHealth / maxHealth) * 100) <= 20):
                healthBar.frame = 3
```

```
                #print("red")
                healthLabel.add_theme_color_override("font_color", Color(255, 0, 0))
        elif(((playerHealth / maxHealth) * 100) <= 30):
                healthBar.frame = 5
        elif(((playerHealth / maxHealth) * 100) <= 40):
                healthBar.frame = 7
        elif(((playerHealth / maxHealth) * 100) <= 50):
                healthBar.frame = 9
                #print("orange")
                healthLabel.add_theme_color_override("font_color", Color(0.898, 0.306, 0.059))
        elif(((playerHealth / maxHealth) * 100) <= 60):
                healthBar.frame = 11
        elif(((playerHealth / maxHealth) * 100) <= 70):
                healthBar.frame = 13
                #print("yellow")
                healthLabel.add_theme_color_override("font_color", Color(255, 255, 0))
        elif(((playerHealth / maxHealth) * 100) <= 80):
                healthBar.frame = 15
        elif(((playerHealth / maxHealth) * 100) <= 90):
                healthBar.frame = 17
        elif(((playerHealth / maxHealth) * 100) <= 100):
                #print("green")
                healthLabel.add_theme_color_override("font_color", Color(0.02, 0.78, 0.02))
```

hud.gd

```
extends CanvasLayer

func update_level(level):
        $LevelLabels/LevelLabel.text = str(level)

func turn_on_cat():
        var soundy = randi() % 100
        print("soundy is: " + str(soundy))
        if(soundy == 20):
                $Hampter.visible = true
                $hampter.play()
                await get_tree().create_timer(10).timeout
                $Hampter.visible = false
        else:
                $cat.visible = true
                $sigma.play()
                await get_tree().create_timer(10).timeout
                $cat.visible = false
```

```
func turn_on_mining():
	$Mining.visible = true
	$LevelLabels.visible = true
```

main.gd

```
extends Node2D

func _on_mine_entrance_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true): #prevents the staticborder from counting as entering the ladder
		get_tree().change_scene_to_file("res://Scenes/Levels/TheFirstMines.tscn")

func _on_ready() -> void:
	$AudioStreamPlayer.play()
	$Player.updateZone(2)
```

MainAreaReturn.gd

```
extends Node2D

func _on_ready() -> void:
	$Player.updateZone(2)
	$darkener.visible = true
	$AnimationPlayer.play("fade in")
	$AudioStreamPlayer.play()
	$"Mine Entrance/CollisionShape2D".set_deferred("disabled", true)
	await get_tree().create_timer(2.0).timeout
	$"Mine Entrance/CollisionShape2D".set_deferred("disabled", false)

func _on_mine_entrance_body_entered(body: Node2D) -> void:
	#await get_tree().create_timer(5.0, true, false, true).timeout #makes a delay
	$AnimationPlayer.play("fade out")
	if(body.is_in_group("PlayerBorders") == false): #prevents the staticborder from counting as entering the ladder
		get_tree().change_scene_to_file("res://Scenes/Levels/TheFirstMines.tscn")

func _on_town_collision_area_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true): #prevents the staticborder from counting as entering the ladder
		$AnimationPlayer.play("fade out")
		get_tree().change_scene_to_file("res://Scenes/Levels/town_area.tscn")
```

mainMenu.gd

```
extends Control


func _on_start_button_pressed() -> void:
	get_tree().change_scene_to_file("res://Scenes/tutorial.tscn")

func _on_continue_pressed() -> void:
	get_tree().change_scene_to_file("res://Scenes/Levels/Main.tscn")

func _on_options_button_pressed() -> void:
	print("Options pressed")
	pass # Replace with function body.


func _on_quit_button_pressed() -> void:
	get_tree().quit()


func _on_button_pressed() -> void:
	#get_tree().change_scene_to_file("res://Scenes/Levels/testZone.tscn")
	get_tree().change_scene_to_file("res://Scenes/Levels/Mines/Mines_1.tscn")
```

main_area_return_return.gd

```
extends Node2D

func _on_ready() -> void:
	$Player.updateZone(2)
	$darkener.visible = true
	$AnimationPlayer.play("fade in")
	$AudioStreamPlayer.play()
	$"Mine Entrance/CollisionShape2D".set_deferred("disabled", true)
	await get_tree().create_timer(2.0).timeout
	$"Mine Entrance/CollisionShape2D".set_deferred("disabled", false)

func _on_mine_entrance_body_entered(body: Node2D) -> void:
	#await get_tree().create_timer(5.0, true, false, true).timeout #makes a delay
	#$AnimationPlayer.play("fade out")
	if(body.is_in_group("PlayerBorders") == false): #prevents the staticborder from counting
as entering the ladder
		get_tree().change_scene_to_file("res://Scenes/Levels/the_mines.tscn")

func _on_town_collision_area_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true): #prevents the staticborder from counting as
entering the ladder
```

```
		#$AnimationPlayer.play("fade out")
		get_tree().change_scene_to_file("res://Scenes/Levels/town_area.tscn")
```

mines_1.gd

```
extends Node2D

#116, 99
#2993,98
#5918, 74
#9352, 20
#10817, -236
#14005, -558
#17708, -81
#19375, 1299
#22044, 860
#24948, -207
var floors = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] #number of floors in that theme - 1 bc it starts at 0
var xCoords = [166, 2993, 5918, 9352, 10817, 14005, 17708, 19375, 22044, 24948] #x coords of where to teleport player
var yCoords = [99, 98, 74, 20, -236, -558, -81, 1299, 860, -207] #y coords of where to teleport player to
var levels = floors.size() #number of floors travelled before next theme
var levelLabelNumber = 0 #what should the levelLabel start at

func _on_ready():
	$HUD.update_level(levelLabelNumber)
	$BlackBackground.visible = true
	$AnimationPlayer.play("fade in")
	print("yes")
	$CaveSounds.play()
	#disbaleExit() #this line NEEDS to be turned on once we decide where we want exit ladders

func disbaleExit():
	if(levels <= 0): #to make it so that once you have entered enough floors, you get sent to the final boos floor
		get_tree().change_scene_to_file("res://Scenes/Levels/the_mines2.tscn")
	else:
		#print("levels is: " + str(levels))
		#print("exit disbaled")
		$ExitLadderr.process_mode = Node.PROCESS_MODE_DISABLED
		await get_tree().create_timer(2.5).timeout
		$ExitLadderr.process_mode = Node.PROCESS_MODE_INHERIT
		#print("exit enable")
```

```
func randomCoords():
	var inty = randi() % floors.size() #levels
	$AnimationPlayer.play("fade out")
	$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
	removey(inty)
	$AnimationPlayer.play("fade in")

func removey(inty):
	levels -= 1
	#print("after removey, levels is: " + str(levels))
	levelLabelNumber +=1
	$HUD.update_level(levelLabelNumber)
	#print("levels is: " + str(levels))
	xCoords.remove_at(inty)
	yCoords.remove_at(inty)
	floors.remove_at(inty)

func _on_exit_ladderr_body_entered(body: Node2D) -> void: #leave mines
	if(body.is_in_group("Player") == true): #prevents the staticborder from counting as
entering the ladder
		$AnimationPlayer.play("fade out")
		await get_tree().create_timer(.5).timeout

get_tree().change_scene_to_file("res://Scenes/Levels/MainAreaReturnReturnReturn.tscn")

func _on_down_ladder_body_entered(body: Node2D) -> void: #go to next floor
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()
```

myAttackRock.gd

```
class_name myAttackRock
extends Area2D

@export var damage := 10

func _init()-> void:
	collision_layer = 3
```

```
	collision_mask = 0
```

myHitbox.gd

```
class_name myHitbox
extends Area2D

@export var damage := 10

func _init()-> void:
	collision_layer = 2
	collision_mask = 0
```

myHurtbox.gd

```
class_name myHurtbox
extends Area2D

func _init() -> void:
	collision_layer = 0
	collision_mask = 2

func _ready() -> void:
	connect("area_entered", self._on_area_entered)

func _on_area_entered(hitbox: myHitbox) -> void:
	if hitbox == null:
		return

	if (owner.has_method("take_damage")):
		owner.take_damage(hitbox.damage)
```

myRockBox.gd

```
class_name myRockbox
extends Area2D

func _init() -> void:
	collision_layer = 0
	collision_mask = 3

func _ready() -> void:
	connect("area_entered", self._on_area_entered)

func _on_area_entered(hitbox: myAttackRock) -> void:
	if hitbox == null:
```

```
		return

	if (owner.has_method("take_damage")):
		owner.take_damage(hitbox.damage)
```

Player.gd

```
extends CharacterBody2D

@export var speed: int = 12000 #400 could be a good run speed #275
@export var knockbackPower: int = 2000

######################################## on ready variable, use @ to make these
@onready var animation = $AnimationPlayer
@onready var boxy = $TheKnight/myHitbox/hitbox #the sword hitbox
@onready var miny = $TheKnight/myAttackRock/RockAttack #the mining hitbox
@onready var healthy = $CanvasLayer #player health bar
@onready var damage_numbers_origin = $DamageNumbersOrigin #where the damage
numbers pop out of the player
@onready var maxHealth: float = 125
@onready var playerHealth: float = maxHealth

########################################
var isAttacking = false #is the player attacking?
var isMining = false #is the player minig?
var notDead = true #is the player alive? used to play the death animation & sound only once per
death (prevents multiple things playing if the player gets hit again after death)
var kindaDead = false #is the player alive? used to allow movement unless the player is dead

############################################################
"""
#reworking movement
func handleInput():
	var moveDirection = Input.get_vector("move-left", "move-right", "move-up",
"move-down")
	velocity = moveDirection * speed

func updateAnimation():
	if(kindaDead == false): #player is alive
		if velocity.length() == 0 && !isAttacking && !isMining:
			animation.play("Idle")
		else:
			var direction = "Down"
			if velocity.x < 0:
				direction = "Left"
```

```
                        $TheKnight.scale.x = -1
                elif velocity.x > 0:
                        direction = "Right"
                        $TheKnight.scale.x = 1
                elif velocity.y < 0:
                        direction = "Up"
                animation.play("Movement")


func _physics_process(delta: float) -> void:
        handleInput()
        move_and_slide()
        updateAnimation()

"""
################################################################

func _on_ready() -> void:
        healthy.visible = true
        healthy.update_health(playerHealth, maxHealth)

        #this currently will just make the player do 20 dmg instead of the default 10,
        #but we can use this format to upgrade the sword damage
        #$TheKnight/myHitbox.damage = 20


func _physics_process(delta: float) -> void:
        velocity.x = 0    #without these, we add the veolcity every frame
        velocity.y = 0    #these lines basically act as friction
        #print(str($".".global_position.x) + ", " + str($".".global_position.y))
        if(kindaDead == false):
                if Input.is_action_pressed("move-left"):
                        if Input.is_action_pressed("run"):
                                velocity.x -= speed * 3 * delta
                                animation.play("Running")
                                $TheKnight.scale.x = -1
                        else:
                                velocity.x -= speed * delta
                                animation.play("Movement")
                                $TheKnight.scale.x = -1
                if Input.is_action_pressed("move-right"):
                        if Input.is_action_pressed("run"):
                                velocity.x += speed * 3 * delta
                                animation.play("Running")
```

```
                $TheKnight.scale.x = 1
            else:
                velocity.x += speed * delta
                animation.play("Movement")
                $TheKnight.scale.x = 1
        if Input.is_action_pressed("move-up"):
            if Input.is_action_pressed("run"):
                velocity.y -= speed * 3 * delta
                animation.play("Running")
            else:
                velocity.y -= speed * delta
                animation.play("Movement")
        if Input.is_action_pressed("move-down"):
            if Input.is_action_pressed("run"):
                velocity.y += speed * 3 * delta
                animation.play("Running")
            else:
                velocity.y += speed * delta
                animation.play("Movement")

        if velocity.x == 0 && velocity.y == 0 && !isAttacking && !isMining:
            animation.play("Idle")

        if Input.is_action_pressed("Pickaxe"): #was right mouse button clicked/held
down?
            animation.play("Mining")
            isMining = true              #then mine
        if Input.is_action_just_released("Pickaxe"): #was left mouse button released?
            isMining = false                 #then stop mining
            miny.disabled = true

        if Input.is_action_pressed("Attack1"): #was left mouse button clicked/held down?
            #velocity.x = 0
            #velocity.y = 0
            animation.play("Attack")
            isAttacking = true              #then attack
        if Input.is_action_just_released("Attack1"): #was left mouse button released?
            isAttacking = false                 #then stop attacking
            boxy.disabled = true

        #for body in $TheKnight/myHitbox.get_overlapping_bodies():
            #if knockbackWait <= 0 and body.is_in_group("Enemies"):
                #print("knockback")
        move_and_slide()   #always have at BOTTOM of code, not top
```

```
		pass


func take_damage(amount: int) -> void: #amount is the damage taken
	#print("Damage: ", amount) #prints the damage taken to the console
	playerHealth -= amount
	if(playerHealth > 0):
		knockback() #gets the enemy velocity to knockback the player
		PlayerDamageNumbersgd.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
		#print("player health is: " + str(playerHealth))
		healthy.update_health(playerHealth, maxHealth)
		$TheKnight/enemyHurtBox.set_deferred("disabled", true)
		#$CollisionShape2D.set_deferred("disabled", true) #makes it so that the player
doesn't get stuck between enemy and fence, but also makes it so that yo ucan just walk right
through the fence??
		await get_tree().create_timer(2).timeout
		$TheKnight/enemyHurtBox.set_deferred("disabled", false)
		#$CollisionShape2D.set_deferred("disabled", false)
	else: #damage is kil
		if(notDead == true): #if the player was not previously dead, do these things
			kindaDead = true
			speed = 0
			$TheKnight/enemyHurtBox/CollisionShape2D2.set_deferred("disabled",
true)
			$deathSounds.play()
			$CanvasLayer.visible = false
			updateZone(1) #removes the HUD from the death animation
			velocity = Vector2(0,0)
			#print("going dark")
			animation.play("ded")
			await get_tree().create_timer(2.0).timeout
			$dieded.dark()
			await get_tree().create_timer(2.0).timeout
			notDead = false
		$DamageNumbersOrigin.visible = false
		get_tree().reload_current_scene() #currently, the entire scene just reloads if the
player dies; can change this to be like Stardew Valley where if the player dies, a hospital scene
is loaded?

func updateZone(zone: int):
	if(zone == 1): #no HUD; used in death animation
		$HUD.visible = false
		#print("zoner 1")
```

```
	elif(zone == 2): #zone above ground; used for the funny secret
		var catty = randi() % 100
		#print("catty is " + str(catty))
		if(catty == 58):
			$HUD.turn_on_cat()
		#print("zone 2")
	#elif(zone == 3): #zone below ground
		#$HUD.turn_on_mining()
		#print("miney time")
	#elif(zone == 4):
		#print("zone 4")

func knockback():
	var knockbackDirection = -velocity.normalized() * knockbackPower
	velocity = knockbackDirection
	move_and_slide()
```

PlayerDamageNumbersgd.gd

```
extends Node


func display_number(value: int, position: Vector2): #Vector2, is_critical: bool = false):
	var number = Label.new()
	number.global_position = position
	number.text = str(value)
	number.z_index = 5
	number.label_settings = LabelSettings.new()

	var color = "#372261"
	#if is_critical:
		#color = "#B22"
	if value == 0:
		color = "#372261"

	number.label_settings.font_color = color
	number.label_settings.font_size = 18
	number.label_settings.outline_color = "#000"
	number.label_settings.outline_size = 1


	call_deferred("add_child", number)

	await number.resized
	number.pivot_offset = Vector2(number.size / 2)
```

```
	var tween = get_tree().create_tween()
	tween.set_parallel(true)
	tween.tween_property(
		number, "position:y", number.position.y - 24, 0.25
	).set_ease(Tween.EASE_OUT)
	tween.tween_property(
		number, "position:y", number.position.y, 0.5
	).set_ease(Tween.EASE_IN).set_delay(0.25)
	tween.tween_property(
		number, "scale", Vector2.ZERO, 0.25
	).set_ease(Tween.EASE_IN).set_delay(0.5)

	await tween.finished
	number.queue_free()
```

pumpkin.gd

```
extends Node2D

@onready var damage_numbers_origin = $DamageNumbersOrigin
@onready var health = 35 #pumpkin health
@onready var notDead = true #box is alive

func _on_ready() -> void:
	$StaticBody2D/collide.set_deferred("disabled", false)

func take_damage(amount: int) -> void: #amount is the damage taken
	#print("Damage: ", amount) #prints the damage taken to the console
	health -= amount
	if(health > 0): #damage not kills the box
		DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
		if(health > 20):
			$PumpkinSpritesheet.frame = 1
		elif(health > 10):
			$PumpkinSpritesheet.frame = 2
		elif(health > 0):
			$PumpkinSpritesheet.frame = 3
	else:
		if(notDead == true):
			DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
			notDead = false
		$PumpkinSpritesheet.visible = false
```

```
		$DamageNumbersOrigin.visible = false
		$StaticBody2D/collide.set_deferred("disabled", true)
```

skeleton_enemy.gd

```
extends Node2D

@onready var damage_numbers_origin = $DamageNumbersOrigin
@onready var SkeletonHealth = 30 #30
@onready var notDead = true #skeleton is alive

func take_damage(amount: int) -> void: #amount is the damage taken
	#print("Damage: ", amount) #prints the damage taken to the console
	SkeletonHealth -= amount
	if(SkeletonHealth > 0): #damage kills the box
		DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
		if (SkeletonHealth < 25):
			$Health3.visible = false
		if (SkeletonHealth < 15):
			$Health2.visible = false
	else: #damage is kil
		if(notDead == true):
			DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
			notDead = false
			$"Skeleton Area2D/SkeletonAnimationPlayer".play("SkeletonDies") #wont
work, idk why???? #also this sprite is ONLY the skeleton, not the whole spritesheet
		#$"Skeleton Area2D/SkeletonSpritesheet".visible = false
		$"Skeleton
Area2D/SkeletonSpritesheet/enemyHitBox/CollisionShape2D".set_deferred("disabled", true)
		$DamageNumbersOrigin.visible = false
		$"Skeleton Area2D/CollisionShape2D".set_deferred("disabled", true)
		$Health1.visible = false

#func _on_skeleton_area_2d_body_entered(body: Node2D) -> void:
	#if (body.is_in_group("Player") && notDead == true):
		###get_tree().change_scene_to_file("res://Scenes/death_scene.tscn")
		##get_tree().reload_current_scene() #when collison body of enemy touches
collision body of player, game restarts
		#print("skeleton entered")
		#
	##pass # Replace with function body.
```

slime_enemy.gd

```
extends CharacterBody2D

@export var patrol_points : Node
@export var speed : int = 1_500
@export var wait_time : int = 3

@onready var animated_sprite_2d = $AnimatedSprite2D
@onready var timer = $Timer

enum State {Idle, Walk}
var current_state : State
var direction : Vector2 = Vector2.RIGHT #enemy moves RIGHT until it falls off the edge
var number_of_points : int
var point_positions : Array[Vector2]
var current_point : Vector2
var current_point_position : int
var can_walk : bool

func _ready():
	if patrol_points != null:
		number_of_points = patrol_points.get_children().size()
		for point in patrol_points.get_children():
			point_positions.append(point.global_position)
		current_point = point_positions[current_point_position]
	else:
		print("No patrol points")

	timer.wait_time = wait_time
	current_state = State.Idle

func _physics_process(delta: float) -> void:
	enemy_idle(delta)
	enemy_walk(delta)

	move_and_slide()

	enemy_animations()

func enemy_idle(delta: float) -> void:
	if !can_walk:
		velocity.x = move_toward(velocity.x, 0, speed * delta)
		#velocity.y = move_toward(velocity.y, 0, speed * delta)
		current_state = State.Idle
```

```
func enemy_walk(delta: float):
	if !can_walk:
		return

	if abs(position.x - current_point.x) > 0.5:
		velocity.x = direction.x * speed * delta
		#velocity.y = direction.y * speed * delta
		current_state = State.Walk
	else:
		current_point_position += 1

		if current_point_position >= number_of_points:
			current_point_position = 0

		current_point = point_positions[current_point_position]

		if current_point.x < position.x:
			direction = Vector2.LEFT
		else:
			direction = Vector2.RIGHT
		can_walk = false
		timer.start()

	animated_sprite_2d.flip_h = direction.x < 0

func enemy_animations():
	if current_state == State.Idle && !can_walk:
		animated_sprite_2d.play("idle")
	elif current_state == State.Walk && can_walk:
		animated_sprite_2d.play("walk")

func _on_timer_timeout() -> void:
	can_walk = true
```

the_first_mines.gd

```
extends Node2D

#116, 99
#2993,98
#5918, 74
#9352, 20
#10817, -236
#14005, -558
```

```
#17708, -81
#19375, 1299
#22044, 860
#24948, -207
var floors = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
var xCoords = [166, 2993, 5918, 9352, 10817, 14005, 17708, 19375, 22044, 24948]
var yCoords = [99, 98, 74, 20, -236, -558, -81, 1299, 860, -207]
var levels = floors.size() #number of floors travlled before final boss floor
var levelLabelNumber = 0

func _on_ready():
	#$Player.updateZone(3)
	#$HUD.turn_on_mining()
	$HUD.update_level(levelLabelNumber)
	$BlackBackground.visible = true
	$AnimationPlayer.play("fade in")
	$CaveSounds.play()
	disbaleExit()

#func _unhandled_input(event: InputEvent) -> void:
	#if event is InputEventKey: #once the exit is enabled again, can press E anywhere and
yeah
		#if event.pressed and event.keycode == KEY_E and $ExitLadderr.process_mode
== Node.PROCESS_MODE_INHERIT:
			#$AnimationPlayer.play("fade out")
			#await get_tree().create_timer(.5).timeout

#get_tree().change_scene_to_file("res://Scenes/Levels/MainAreaReturn.tscn")
	#if event is InputEventKey: #pauses but won't unpause
		#if event.pressed and event.keycode == KEY_ESCAPE and get_tree().paused
== false:
			#get_tree().paused = true
		#elif event.pressed and event.keycode == KEY_ESCAPE and get_tree().paused
== true:
			#get_tree().paused = false

func disbaleExit():
	if(levelLabelNumber == 4):
		$Player/GoHomeLabel.visible = true
		$ExitLadderr.process_mode = Node.PROCESS_MODE_DISABLED
		await get_tree().create_timer(2.5).timeout
		$ExitLadderr.process_mode = Node.PROCESS_MODE_INHERIT
		print("exit enable")
```

```
	elif(levelLabelNumber == 5): #to make it so that once you have entered enough floors, you get sent to the final boos floor
		print("yes")
		get_tree().change_scene_to_file("res://Scenes/Levels/MainAreaReturn.tscn")
	else:
		print("levels is: " + str(levels))
		print("exit disbaled")
		$ExitLadderr.process_mode = Node.PROCESS_MODE_DISABLED
		await get_tree().create_timer(2.5).timeout
		$ExitLadderr.process_mode = Node.PROCESS_MODE_INHERIT
		print("exit enable")

func randomCoords():
	var inty = randi() % floors.size() #levels
	#print("inty is:" + str(inty))
	#print(xCoords[inty])
	#print(yCoords[inty])
	#print("-----")
	#print(xCoords)
	#print(yCoords)
	#print(floors)
	#print("------------------")
	$AnimationPlayer.play("fade out")
	$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
	removey(inty)
	$AnimationPlayer.play("fade in")

func removey(inty):
	levels -= 1
	#print("after removey, levels is: " + str(levels))
	levelLabelNumber +=1
	$HUD.update_level(levelLabelNumber)
	#print("levels is: " + str(levels))
	xCoords.remove_at(inty)
	yCoords.remove_at(inty)
	floors.remove_at(inty)

func _on_exit_ladderr_body_entered(body: Node2D) -> void: #leave mines
	if(body.is_in_group("Player") == true):
		$AnimationPlayer.play("fade out")
		await get_tree().create_timer(.5).timeout
		get_tree().change_scene_to_file("res://Scenes/Levels/MainAreaReturn.tscn")

func _on_down_ladder_body_entered(body: Node2D) -> void: #go to next floor
```

```
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_2_body_entered(body: Node2D) -> void: #go to next floor
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_3_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_4_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_5_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
```

```
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()

func _on_down_ladder_6_body_entered(body: Node2D) -> void:
        if(body.is_in_group("Player") == true):
                #var inty = randomCoords()
                randomCoords()
                #$AnimationPlayer.play("fade out")
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()

func _on_down_ladder_7_body_entered(body: Node2D) -> void:
        if(body.is_in_group("Player") == true):
                #var inty = randomCoords()
                randomCoords()
                #$AnimationPlayer.play("fade out")
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()

func _on_down_ladder_8_body_entered(body: Node2D) -> void:
        if(body.is_in_group("Player") == true):
                #var inty = randomCoords()
                randomCoords()
                #$AnimationPlayer.play("fade out")
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()

func _on_down_ladder_9_body_entered(body: Node2D) -> void:
        if(body.is_in_group("Player") == true):
                #var inty = randomCoords()
                randomCoords()
                #$AnimationPlayer.play("fade out")
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()
```

```
func _on_down_ladder_10_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_11_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()
```

the_mines.gd

```
extends Node2D

#116, 99
#2993,98
#5918, 74
#9352, 20
#10817, -236
#14005, -558
#17708, -81
#19375, 1299
#22044, 860
#24948, -207
var floors = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] #number of floors in that theme - 1 bc it starts at 0
var xCoords = [166, 2993, 5918, 9352, 10817, 14005, 17708, 19375, 22044, 24948] #x coords of where to teleport player
var yCoords = [99, 98, 74, 20, -236, -558, -81, 1299, 860, -207] #y coords of where to teleport player to
var levels = floors.size() #number of floors travelled before next theme
var levelLabelNumber = 0 #what should the levelLabel start at

func _on_ready():
	#$Player.updateZone(3)
```

```
	#print("levelLabelNumber of mines1 is: " + str(levelLabelNumber))
	#$HUD.turn_on_mining()
	$HUD.update_level(levelLabelNumber)
	$BlackBackground.visible = true
	$AnimationPlayer.play("fade in")
	$CaveSounds.play()
	disbaleExit()

func disbaleExit():
	if(levels <= 0): #to make it so that once you have entered enough floors, you get sent to the final boos floor
		get_tree().change_scene_to_file("res://Scenes/Levels/the_mines2.tscn")
	else:
		#print("levels is: " + str(levels))
		#print("exit disbaled")
		$ExitLadderr.process_mode = Node.PROCESS_MODE_DISABLED
		await get_tree().create_timer(2.5).timeout
		$ExitLadderr.process_mode = Node.PROCESS_MODE_INHERIT
		#print("exit enable")

func randomCoords():
	var inty = randi() % floors.size() #levels
	#print("inty is:" + str(inty))
	#print(xCoords[inty])
	#print(yCoords[inty])
	#print("-----")
	#print(xCoords)
	#print(yCoords)
	#print(floors)
	#print("------------------")
	$AnimationPlayer.play("fade out")
	$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
	removey(inty)
	$AnimationPlayer.play("fade in")

func removey(inty):
	levels -= 1
	#print("after removey, levels is: " + str(levels))
	levelLabelNumber +=1
	$HUD.update_level(levelLabelNumber)
	#print("levels is: " + str(levels))
	xCoords.remove_at(inty)
	yCoords.remove_at(inty)
	floors.remove_at(inty)
```

```
func _on_exit_ladderr_body_entered(body: Node2D) -> void: #leave mines
	if(body.is_in_group("Player") == true): #prevents the staticborder from counting as entering the ladder
		$AnimationPlayer.play("fade out")
		await get_tree().create_timer(.5).timeout

get_tree().change_scene_to_file("res://Scenes/Levels/MainAreaReturnReturnReturn.tscn")

func _on_down_ladder_body_entered(body: Node2D) -> void: #go to next floor
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_2_body_entered(body: Node2D) -> void: #go to next floor
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_3_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_4_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
```

```
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()

func _on_down_ladder_5_body_entered(body: Node2D) -> void:
        if(body.is_in_group("Player") == true):
                #var inty = randomCoords()
                randomCoords()
                #$AnimationPlayer.play("fade out")
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()

func _on_down_ladder_6_body_entered(body: Node2D) -> void:
        if(body.is_in_group("Player") == true):
                #var inty = randomCoords()
                randomCoords()
                #$AnimationPlayer.play("fade out")
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()

func _on_down_ladder_7_body_entered(body: Node2D) -> void:
        if(body.is_in_group("Player") == true):
                #var inty = randomCoords()
                randomCoords()
                #$AnimationPlayer.play("fade out")
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()

func _on_down_ladder_8_body_entered(body: Node2D) -> void:
        if(body.is_in_group("Player") == true):
                #var inty = randomCoords()
                randomCoords()
                #$AnimationPlayer.play("fade out")
                #$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
                #removey(inty)
                #$AnimationPlayer.play("fade in")
                disbaleExit()
```

```
func _on_down_ladder_9_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_10_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()

func _on_down_ladder_11_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		#var inty = randomCoords()
		randomCoords()
		#$AnimationPlayer.play("fade out")
		#$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		#removey(inty)
		#$AnimationPlayer.play("fade in")
		disbaleExit()
```

the_mines_2.gd

```
extends Node2D

#116, 99
#2993,98
#5918, 74
#9352, 20
#10817, -236
#14005, -558
#19375, 1299
#22044, 860
#24948, -207
#17708, -81
```

```
var floors = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  #number of floors in that theme - 1 bc it starts at 0
var xCoords = [166, 2993, 5918, 9352, 10817, 14005, 17708, 19375, 22044, 24948] #x coords of where to teleport player
var yCoords = [99, 98, 74, 20, -236, -558, -81, 1299, 860, -207] #y coords of where to teleport player to
var levels = floors.size() #number of floors travelled before next theme
var levelLabelNumber = 10 #what should the levelLabel start at

func _on_ready():
	#$Player.updateZone(3)
	#print("levelLabelNumber is: " + str(levelLabelNumber))
	#$HUD.turn_on_mining()
	#print("turned on mining")
	$HUD.update_level(levelLabelNumber)
	$BlackBackground.visible = true
	$AnimationPlayer.play("fade in")
	$CaveSounds.play()
	disbaleExit()

func disbaleExit():
	if(levels <= 0): #to make it so that once you have entered enough floors, you get sent to the final boos floor
		print("levels = o")
		levels = 0
		get_tree().change_scene_to_file("res://Scenes/Levels/the_mines2.tscn") #this will be like mines3, NOT mines2
	else:
		print("levels is: " + str(levels))
		#print("exit disbaled")
		$ExitLadderr.process_mode = Node.PROCESS_MODE_DISABLED
		await get_tree().create_timer(2.5).timeout
		$ExitLadderr.process_mode = Node.PROCESS_MODE_INHERIT
		#print("exit enable")

func randomCoords():
	var inty = randi() % floors.size() #levels
	#print("inty is:" + str(inty))
	#print(xCoords[inty])
	#print(yCoords[inty])
	#print("-----")
	#print(xCoords)
	#print(yCoords)
	#print(floors)
	#print("------------------")
```

```
		$AnimationPlayer.play("fade out")
		$Player.set_position(Vector2(xCoords[inty], yCoords[inty]))
		removey(inty)
		$AnimationPlayer.play("fade in")

func removey(inty):
		levels -= 1
		print("after removey, levels is: " + str(levels))
		levelLabelNumber +=1
		$HUD.update_level(levelLabelNumber)
		#print("levels is: " + str(levels))
		xCoords.remove_at(inty)
		yCoords.remove_at(inty)
		floors.remove_at(inty)

func _on_exit_ladderr_body_entered(body: Node2D) -> void: #leave mines
		if(body.is_in_group("Player") == true): #prevents the staticborder from counting as
entering the ladder
				$AnimationPlayer.play("fade out")
				await get_tree().create_timer(.5).timeout

get_tree().change_scene_to_file("res://Scenes/Levels/MainAreaReturnReturnReturn.tscn")
```

town_area.gd

```
extends Node2D

func _on_ready() -> void:
		$Player.updateZone(1) #no HUD instructions
		$AudioStreamPlayer2.play()

func _on_audio_stream_player_finished() -> void:
		$AudioStreamPlayer.play()

func _on_audio_stream_player_2_finished() -> void:
		$AudioStreamPlayer2.play()

func _on_area_2d_2_body_entered(body: Node2D) -> void: #bridge
		if(body.is_in_group("Player") == true):
				$AudioStreamPlayer2.stop()
				$AudioStreamPlayer.play()

func _on_area_2d_2_body_exited(body: Node2D) -> void: #bridge
		$AudioStreamPlayer.stop()
		$AudioStreamPlayer2.play()
```

```
func _on_area_2d_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true): #only if the player collides with this box will the scene change

get_tree().change_scene_to_file("res://Scenes/Levels/MainAreaReturnReturn.tscn")
```

tutorial.gd

```
extends Node2D

func _on_ready() -> void:
	$Player.updateZone(1)

func _on_area_2d_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		get_tree().change_scene_to_file("res://Scenes/Levels/Main.tscn")

func _on_w_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		$w/Node2D.visible = false
		$w/Node2D2.visible = true

func _on_s_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		$w/Node2D2.visible = false
		$s/Node2D.visible = true

func _on_a_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		$s/Node2D.visible = false
		$a/Node2D.visible = true

func _on_pumpin_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		$a/Node2D.visible = false
		$pumpin.visible = true
		$AnimationPlayer.play("attack")

func _on_rocck_body_entered(body: Node2D) -> void:
	if(body.is_in_group("Player") == true):
		$pumpin.visible = false
		$rocck.visible = true
		$AnimationPlayer.play("mine")
```

waves.gd

```
extends CharacterBody2D

@export var patrol_points : Node
@export var speed : int = 1_500

enum State {Idle, Walk}
var current_state : State
var direction : Vector2 = Vector2.RIGHT #enemy moves RIGHT until it falls off the edge
var number_of_points : int
var point_positions : Array[Vector2]
var current_point : Vector2
var current_point_position : int

func _ready():
	if patrol_points != null:
		number_of_points = patrol_points.get_children().size()
		for point in patrol_points.get_children():
			point_positions.append(point.global_position)
		current_point = point_positions[current_point_position]
	else:
		print("No patrol points")


func _physics_process(delta: float) -> void:
	enemy_walk(delta)
	move_and_slide()

func enemy_walk(delta: float):
	if abs(position.x - current_point.x) > 0.5:
		velocity.x = direction.x * speed * delta
		current_state = State.Walk
	else:
		current_point_position += 1

		if current_point_position >= number_of_points:
			current_point_position = 0

		current_point = point_positions[current_point_position]

		if current_point.x < position.x:
			direction = Vector2.LEFT
		else:
			direction = Vector2.RIGHT
```

wraith_enemy.gd

```
extends Node2D

@onready var animation_player := $"Wraith Area2D/WraithAnimationPlayer"

@onready var damage_numbers_origin = $DamageNumbersOrigin
@onready var WraithHealth = 50 #50
@onready var notDead = true #wraith is alive

func take_damage(amount: int) -> void: #amount is the damage taken
	#print("Damage: ", amount) #prints the damage taken to the console
	WraithHealth -= amount
	if(WraithHealth > 0): #damage not kill
		DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
	else: #damage is kill
		if(notDead == true):
			DamageNumbers.display_number(amount,
damage_numbers_origin.global_position) #, is_critical
			notDead = false
			$"Wraith Area2D/WraithAnimationPlayer".play("WraithDies") #wont work,
idk why????
		#$"Wraith Area2D/WraithSpritesheet".visible = false
		$"Wraith
Area2D/WraithSpritesheet/enemyHitBox/CollisionShape2D".set_deferred("disabled", true)
		$DamageNumbersOrigin.visible = false
		$"Wraith Area2D/CollisionShape2D".set_deferred("disabled", true)


#func _on_wraith_area_2d_body_entered(body: Node2D) -> void:
	#if (body.is_in_group("Player") && notDead == true):
		###get_tree().change_scene_to_file("res://Scenes/death_scene.tscn")
		##get_tree().reload_current_scene() #when collison body of enemy touches
collision body of player, game restarts
		#print("wraith entered")
	##pass # Replace with function body.
```